INSTITUT TEKNOLOGI DEL

MATERI TUGAS

Pemrograman Berorientasi Objek

SEMESTER GANJIL T.A 2026/2027

Tanggal / Sesi	: Jumat, 2 Oktober 2026
Semester	: Ganjil
Mata Kuliah	: Pemrograman Berorientasi Objek
Minggu/Sesi	: 04/ 03
Aktivitas	: Encapsulation, Packages, and Interfaces
Durasi	: 110 Menit
Setoran	: Softfile
Batas Pengumpulan	: Akhir Sesi
Tujuan	: Mahasiswa mampu menerapkan Encapsulation, Packages, and Interfaces

Bagian A. Praktikum Terpandu ----------

Praktek 1 – Sistem Keamanan Rekening Bank

Latar Belakang Masalah: Pada sebuah sistem perbankan digital, setiap nasabah memiliki data rekening yang memuat nomor akun dan saldo tabungan. Nilai saldo sangat sensitif dan rawan terjadi human error atau manipulasi jika bisa diubah secara bebas dari luar kelas. Oleh karena itu, saldo tidak boleh bernilai negatif dan transaksi penarikan tidak boleh melebihi saldo yang tersedia.

Tugas: Buat program Java untuk mengelola data rekening menggunakan prinsip enkapsulasi. Gunakan hak akses `private` pada variabel rekening dan saldo. Sediakan metode getter untuk membaca saldo, serta metode setter/mutator untuk melakukan transaksi setor (deposit) dan tarik tunai (withdraw). Tambahkan logika validasi di dalam metode tersebut untuk memastikan jumlah setoran selalu positif dan penarikan tidak menyebabkan saldo minus.

```
public class BankAccount {
    private String accountNumber;
    private double balance;

    public BankAccount(String accountNumber, double initialBalance) {
        this.accountNumber = accountNumber;
        if (initialBalance >= 0) {
            this.balance = initialBalance;
        } else {
            System.out.println("Saldo awal tidak boleh negatif! Diatur ke 0.");
            this.balance = 0;
        }
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public double getBalance() {
        return balance;
    }

    public void deposit(double amount) {
        if (amount > 0) {
            this.balance += amount;
            System.out.println("Setor berhasil: Rp" + amount);
        } else {
            System.out.println("Jumlah setoran harus lebih besar dari 0!");
        }
    }

    public void withdraw(double amount) {
        if (amount <= 0) {
            System.out.println("Jumlah penarikan tidak valid!");
        } else if (amount > this.balance) {
            System.out.println("Penarikan gagal: Saldo tidak mencukupi!");
        } else {
            this.balance -= amount;
            System.out.println("Tarik tunai berhasil: Rp" + amount);
        }
    }
```

```
    public static void main(String[] args) {
        BankAccount acc = new BankAccount("ACC-001", 500000);

        acc.deposit(-100000);
        acc.withdraw(700000);
        acc.withdraw(200000);

        System.out.println("Saldo akhir akun " + acc.getAccountNumber() + ": Rp" + acc.getBalance());
    }
}
```

Output:

```
run:
Jumlah setoran harus lebih besar dari 0!
Penarikan gagal: Saldo tidak mencukupi!
Tarik tunai berhasil: Rp200000.0
Saldo akhir akun ACC-001: Rp300000.0
```

Praktek 2 – Sistem Pembayaran Digital

Latar Belakang Masalah: Pada aplikasi marketplace modern, platform harus mendukung berbagai metode pembayaran seperti dompet digital (E-Wallet) dan kartu kredit. Meskipun cara kerja teknis dari masing-masing metode berbeda di belakang layar, modul kasir (checkout) membutuhkan standar yang seragam agar proses pembayaran apa pun bisa dipicu dengan perintah yang sama tanpa menulis ulang alur transaksi.

Tugas: Buat program Java menggunakan *interface* untuk menstandarisasi proses transaksi pembayaran. Definisikan sebuah *interface* `PaymentMethod` yang memuat konstanta biaya admin dan deklarasi metode pembayaran. Buat dua kelas implementasi (`EWalletPayment` dan `CreditCardPayment`) yang mewarisi kontrak interface tersebut dengan logika pemrosesan masing-masing. Jalankan transaksi di kelas utama menggunakan referensi dari tipe interface tersebut.

```java
interface PaymentMethod {
    double TRANSACTION_FEE = 2500.0;

    void pay(double amount);
    String getPaymentDetails();
}

class EWalletPayment implements PaymentMethod {
    private String phoneNumber;

    public EWalletPayment(String phoneNumber) {
        this.phoneNumber = phoneNumber;
    }

    @Override
    public void pay(double amount) {
        double total = amount + TRANSACTION_FEE;
        System.out.println("[E-Wallet " + phoneNumber + "] Pembayaran diproses: Rp" + amount
                           + " (Biaya admin: Rp" + TRANSACTION_FEE + ") | Total: Rp" + total);
    }

    @Override
    public String getPaymentDetails() {
        return "Metode: E-Wallet (" + phoneNumber + ")";
    }
}

class CreditCardPayment implements PaymentMethod {
    private String cardNumber;

    public CreditCardPayment(String cardNumber) {
        this.cardNumber = cardNumber;
    }

    @Override
    public void pay(double amount) {
        System.out.println("[Credit Card " + cardNumber + "] Pembayaran berhasil diverifikasi sebesar: Rp" + amount);
    }

    @Override
    public String getPaymentDetails() {
        return "Metode: Kartu Kredit (" + cardNumber + ")";
    }
}
```

```java
public class PaymentSystem {
    public static void main(String[] args) {
        PaymentMethod payment1 = new EWalletPayment("08123456789");
        PaymentMethod payment2 = new CreditCardPayment("4111-XXXX-XXXX-1234");

        payment1.pay(50000);
        payment2.pay(150000);
    }
}
```

Output:

```
run:
[E-Wallet 08123456789] Pembayaran diproses: Rp50000.0 (Biaya admin: Rp2500.0) | Total: Rp52500.0
[Credit Card 4111-XXXX-XXXX-1234] Pembayaran berhasil diverifikasi sebesar: Rp150000.0
```

Bagian B. Latihan Mandiri ----------

Semua latihan di bawah ini, juga dilatarbelakangi dari masalah nyata:

1. **Sistem validasi Nilai Mahasiswa**
   Pada portal akademik kampus, data nilai mahasiswa rawan mengalami kesalahan input (human error) oleh staf pengajar jika tidak dibatasi nilainya. Selain itu, nomor induk mahasiswa (NIM) dan nama tidak boleh diubah sembarangan setelah data dibuat agar tidak merusak arsip.

   Tugas: Buat program Java menggunakan kelas `StudentGrade` untuk mencatat nilai kuis dan nilai UTS mahasiswa. Kunci semua atribut menggunakan hak akses `private`. Terapkan sifat *read-only* (hanya sediakan *getter*, tanpa *setter*) untuk data NIM dan Nama. Buat metode *setter* untuk nilai kuis dan UTS yang memvalidasi bahwa input nilai harus berada pada rentang **0.0 sampai 100.0**; jika di luar rentang tersebut, tolak perubahan dan tampilkan pesan kesalahan. Tambahkan method untuk menghitung nilai akhir (bobot: 40% Kuis + 60% UTS) dan cetak laporan hasil studinya.

   Tantangan Tambahan (State & Read-Only Property):
   Tambahkan satu atribut read-only (hanya memiliki getter) bernama `gradeHuruf` *(bertipe String: A, B, C, D, atau E). Nilai* `gradeHuruf` *tidak boleh di-set secara langsung dari kelas utama, melainkan harus terkalkulasi secara otomatis menggunakan internal method yang bersifat private setiap kali setter nilai kuis atau nilai UTS berhasil dipanggil*

   Output (contoh):

```
run:
[Registrasi] Berhasil mendaftarkan mahasiswa: 11425001 - Marsha
[Error] Nilai kuis tidak valid: 110.0 (Harus antara 0 - 100)
[Update] Nilai kuis berhasil diubah menjadi: 85.0
[Update] Nilai UTS berhasil diubah menjadi: 90.0
=== KARTU HASIL STUDI ===
NIM        : 11425001
Nama       : Marsha
Nilai Kuis : 85.0
Nilai UTS  : 90.0
Nilai Akhir: 88.0
```

2. **Pengendalian Perangkat Smart Home**
   Dalam sistem rumah pintar (smart home), setiap perangkat elektronik seperti lampu pintar dan AC memiliki fungsi mekanis yang berbeda, namun keduanya sama-sama membutuhkan kemampuan sakelar (daya nyala/mati) serta modul komunikasi nirkabel untuk terhubung ke jaringan internet rumah.

   Tugas: Buat program Java dengan memanfaatkan konsep antarmuka (*interface*). Buat interface `Switchable` dengan metode `turnOn()` dan `turnOff()`. Buat interface `Connectable` dengan metode `connectToWifi(String ssid, String ipAddress)`. Buat

kelas `SmartLamp` dan `SmartAC` yang keduanya mengimplementasikan kedua interface tersebut secara bersamaan (`implements Switchable, Connectable`). Bungkus atribut internal perangkat (seperti persentase kecerahan dan derajat suhu) secara privat, lalu simulasikan proses menyalakan perangkat, menghubungkannya ke WiFi, dan mematikannya melalui metode `main`.

Tambahan Aturan Logika (State Management):
Integrasikan logika status (State). Sebuah smart device tidak diizinkan menjalankan fungsi connectToWifi(ssid, ipAddress) dari interface Connectable jika status sakelar perangkat tersebut masih dalam keadaan mati (belum dipanggil turnOn()). Berikan pesan error jika perangkat mencoba terkoneksi WiFi dalam keadaan mati.

Output (contoh):

```
run:
[Lampu Pintar] Dinyalakan dengan kecerahan 80%
[WiFi Device] Terhubung ke jaringan: Rumah_Indah_5G (IP: 192.168.1.50)
[AC Pintar] Dinyalakan pada suhu 22 C
[WiFi Device] Terhubung ke jaringan: Rumah_Indah_5G (IP: 192.168.1.51)
[AC Pintar] Dimatikan.
```

Bagian C. Kelas Interaktif ----------

Sebuah tim pengembang menulis kode sistem reservasi kendaraan berikut.
Analisis rancangannya bersama rekan sekelompok Anda:

```
package id.ac.del.rental;

public class VehicleRental {
    public String plateNumber;
    protected double rentalRatePerDay;
    private int rentalDays;
    double discountRate;

    public VehicleRental(String plateNumber, double rate) {
        this.plateNumber = plateNumber;
        this.rentalRatePerDay = rate;
    }

    public void setRentalDays(int days) {
        this.rentalDays = days;
    }

    public double getTotalCost() {
        return (rentalRatePerDay * rentalDays) -
               (rentalRatePerDay * rentalDays * discountRate);
    }
}
```

```
package id.ac.del.test;

import id.ac.del.rental.VehicleRental;

public class TestRental {
    public static void main(String[] args) {
        VehicleRental v = new VehicleRental("BK 1234 DEL", 250000);

        // Baris A
        v.plateNumber = "";

        // Baris B
        v.rentalRatePerDay = -500000;

        // Baris C
        v.discountRate = 0.20;

        // Baris D
        v.setRentalDays(-7);

        System.out.println("Total Biaya: Rp" + v.getTotalCost());
    }
}
```

Silahkan diskusikan bersama kelompok belajar Anda.

1) Analisis Kompilasi dan Hak Akses (Access Modifiers)
    i) Saat `TestRental.java` dikompilasi, baris manakah di antara **Baris A, B, C, dan D** yang akan memicu *compile-time error* karena perbedaan package? Jelaskan aturan cakupan akses (`public`, `protected`, `default`, dan `private`) yang menyebabkannya.
2) Kelemahan Integritas Data (Data Hiding & Validation)
    i) Perhatikan **Baris A** dan **Baris D**. Walaupun Baris D memanggil metode setter, apa cacat logika yang masih terjadi pada method `setRentalDays`?
    ii) Nilai total biaya yang dihasilkan bisa bernilai negatif. Mengapa hal ini bisa terjadi dan bagaimana cara merefaktor kelas `VehicleRental` agar data keuangannya aman dari korupsi data (*data corruption*)?
3) Penerapan Interface (Dependency Injection)
   Jika Sistem ini akan dikembangkan untuk mendukung berbagai skema diskon (misalnya `MemberDiscount`, `FlashSaleDiscount`, dan `CorporateDiscount`):
    i) Rancanglah kontrak `interface Discountable` yang memiliki metode `calculateDiscount(double amount)`.
    ii) Ubah dan tuliskan ulang kode metode `getTotalCost()` pada kelas `VehicleRental` agar menerima parameter bertipe antarmuka `Discountable`. Supaya perhitungan total biaya dapat langsung mengenali dan memotong harga sesuai dengan objek diskon yang dikirimkan dari kelas `TestRental` saat *runtime* (polimorfisme).

Format Pengumpulan:

- Kumpulkan dalam bentuk **folder ZIP** dengan nama: `NIM_PBO_Sesi3.zip` (contoh: `22123456_PBO_Sesi3.zip`)
- **Wajib ada komentar pada setiap program** untuk menjelaskan baris kode atau blok logika atau penjelasan fungsi program.